

CS 473 (Spring 2025)

Homework 5 Solutions

Problem 5.1: In this problem, we will explore other simple families of hash functions and see how close they are to being universal. Let $U \geq m$.

- (a) (35 pts) Let p be a random prime in $[m/2, m]$. Let b be a random number in $\{0, \dots, p-1\}$. Define $h_p : \{0, \dots, U-1\} \rightarrow \{0, \dots, m-1\}$ by

$$h_p(x) = x \bmod p.$$

Prove that for any fixed $x, y \in \{0, \dots, U-1\}$ with $x \neq y$, we have $\Pr_p[h_p(x) = h_p(y)] \leq O((\log U)/m)$.

(Thus, this hash function family is “almost” universal with an extra logarithmic factor.)

[Hint: By the prime number theorem, the number of primes in $[m/2, m]$ is $\Theta(m/\log m)$. Given a number $z \leq U$, how many prime factors in $[m/2, m]$ can z have?]

- (b) (35 pts) Suppose $U = 2^w$ and $m = 2^\ell$. We can view numbers in $\{0, \dots, U-1\}$ as binary vectors in $\{0, 1\}^w$, and numbers in $\{0, \dots, m-1\}$ as binary vectors in $\{0, 1\}^\ell$.

Pick a random 0-1 matrix A of dimension $\ell \times w$ (each of its ℓw entries is an independently chosen random bit). Define $h_A : \{0, 1\}^w \rightarrow \{0, 1\}^\ell$ by

$$h_A(x) = (Ax) \bmod 2.$$

All arithmetic operations are done modulo 2; for example, addition is equivalent to exclusive-or.

Prove that for any fixed $x, y \in \{0, 1\}^w$ with $x \neq y$, we have $\Pr_A[h_A(x) = h_A(y)] = 1/m$.

(Thus, this hash function family is universal, and its computation avoids integer division and requires only bitwise exclusive-or, which is cheaper. However, the evaluation time isn't $O(1)$.)

[Hint: suppose x and y differs at the i -th bit. Focus on choices over the i -th column of A ...]

- (c) (30 pts) Continuing part (b), let b be a random 0-1 vector of dimension ℓ . Define $h_{A,b} : \{0, 1\}^w \rightarrow \{0, 1\}^\ell$ by

$$h_{A,b}(x) = (Ax + b) \bmod 2.$$

Prove that for any fixed $x, y \in \{0, 1\}^w$ with $x \neq y$ and for any fixed $s, t \in \{0, 1\}^\ell$, we have $\Pr_{A,b}[(h_{A,b}(x) = s) \wedge (h_{A,b}(y) = t)] = 1/m^2$.

(Thus, this hash function family is 2-universal.)

[Hint: use (b).]

Solution:

- (a) For a fixed $x, y \in \{0, 1, \dots, U-1\}$ with $x \neq y$,

$$\begin{aligned} \Pr_p[h_p(x) = h_p(y)] &= \Pr_p[x \bmod p = y \bmod p] \\ &= \Pr_p[p \text{ is a prime divisor of } x - y] \\ &= \frac{\text{number of distinct prime divisors of } x - y \text{ in } [m/2, m]}{k}, \end{aligned}$$

where k is the number of primes in $[m/2, m]$. By the prime number theorem, $k = \Theta(m/\log m)$.

Note that if $x - y$ has t distinct prime divisors $p_1, p_2, \dots, p_t \in [m/2, m]$, then $|x - y| \geq p_1 p_2 \cdots p_t \geq (m/2)^t$, implying that $t \leq \log_{m/2} |x - y| \leq \log_{m/2} U$.

It follows that

$$\Pr_p[h_p(x) = h_p(y)] \leq \frac{\log_{m/2} U}{k} = O\left(\frac{\log U / \log m}{m / \log m}\right) = O\left(\frac{\log U}{m}\right).$$

- (b) We are given a fixed $x, y \in \{0, 1, \dots, U-1\}$ with $x \neq y$. Let $z = x - y$. Let z_i be the i -th element of z . Let $v_i \in \{0, 1\}^\ell$ be the i -th column vector of A . Since z is nonzero, we can pick an index i_0 with $z_{i_0} = 1$.

$$\begin{aligned} \Pr_A[h_A(x) = h_A(y)] &= \Pr_A[Ax \equiv Ay \pmod{2}] \\ &= \Pr_A[Az \equiv 0 \pmod{2}] \\ &= \Pr_A[v_1 z_1 + \cdots + v_w z_w \equiv 0 \pmod{2}] \\ &= \Pr_A\left[v_{i_0} \equiv \sum_{j \in \{1, \dots, w\} - \{i_0\}} v_j z_j \pmod{2}\right]. \end{aligned}$$

Conditioned on fixed choices of all v_j with $j \neq i_0$, the right-hand side is fixed. The probability that v_{i_0} is equal to a specific vector is $1/2^\ell$. Thus, the probability that $v_{i_0} = \sum_{j \in \{1, \dots, w\} - \{i_0\}} v_j z_j \pmod{2}$ is $1/2^\ell$, unconditionally. We conclude that $\Pr_A[h_A(x) = h_A(y)] = 1/2^\ell$.

[*Remark.* Alternatively, we can directly count the number of matrices A satisfying $Az \equiv 0$. Let T be the subset of indices at which z is nonzero. Note that $|T| \geq 1$. We want each row of A to have an even number of 1's in the positions of T . For each row, there are $2^{|T|-1}$ choices of bits in those positions satisfying the evenness property (half of all $2^{|T|}$ possibilities), and there are $2^{w-|T|}$ choices for the remaining bits. So, there are 2^{w-1} choices for each row. The total number of matrices A satisfying $Az \equiv 0$ is thus $(2^{w-1})^\ell$. Hence, the probability is $\frac{(2^{w-1})^\ell}{2^{w\ell}} = 1/2^\ell = 1/m$.]

- (c) We are given a fixed $x, y \in \{0, 1, \dots, U-1\}$ with $x \neq y$, and given a fixed $s, t \in \{0, 1\}^\ell$. Let $z = x - y$ and $r = s - t$. Let z_i be the i -th element of z . Let $v_i \in \{0, 1\}^\ell$ be the i -th column vector of A . Since z is nonzero, we can pick an index i_0 with $z_{i_0} = 1$.

$$\begin{aligned}
& \Pr_{A,b}[(h_{A,b}(x) = s) \wedge h_{A,b}(y) = t)] \\
&= \Pr_{A,b}[(Ax + b \equiv s) \wedge (Ay + b \equiv t) \pmod{2}] \\
&= \Pr_{A,b}[(Az \equiv r) \wedge (b \equiv s - Ax) \pmod{2}] \\
&= \Pr_{A,b} \left[\left(v_{i_0} \equiv \sum_{j \in \{1, \dots, w\} - \{i_0\}} v_j z_j + r \right) \wedge (b \equiv s - Ax) \pmod{2} \right]
\end{aligned}$$

Conditioned on fixed choices of all v_j with $j \neq i_0$, $\sum_{j \in \{1, \dots, w\} - \{i_0\}} v_j z_j + r$ is fixed. The probability that v_{i_0} is equal to this vector is $1/2^\ell = 1/m$.

Conditioned on a fixed choice of $A = (v_1 \mid \dots \mid v_w)$, the probability that $b \equiv s - Ax \pmod{2}$ is $1/2^\ell = 1/m$.

We conclude that $\Pr_{A,b}[(h_{A,b}(x) = s) \wedge h_{A,b}(y) = t)] = 1/m \cdot 1/m = 1/m^2$.

[*Remark.* Alternatively, we can again use a direct counting argument. The number of matrices A satisfying $Az \equiv r$ is $(2^{w-1})^\ell$ as before. For each such A , the number of vectors b satisfying $b \equiv s - Ax$ is one. Hence, the probability is $\frac{(2^{w-1})^\ell \cdot 1}{2^{w\ell} \cdot 2^\ell} = 1/2^{2\ell} = 1/m^2$.]

Problem 5.2: We are given a subset S of $\{1, \dots, N\}$ and a number $k \leq N$. We want to compute a new set $f_k(S) = \{a_1 + \dots + a_k : a_1, \dots, a_k \text{ are } k \text{ distinct elements of } S\}$.

(Note that this is a little different from one of the midterm 1 practice problems, which is about computing the set $S + \dots + S$ with k terms, where an element may be used more than once in the sum.)

- (a) (50 pts) First consider a related problem: given ℓ disjoint sets $S_1, \dots, S_\ell \subseteq \{1, \dots, N\}$ and a number $k \leq \ell \leq N$, compute a new set $g_k(S_1, \dots, S_\ell) = \{a_1 + \dots + a_k : a_1, \dots, a_k \text{ are elements chosen from } k \text{ different sets among } S_1, \dots, S_\ell\}$.

Give a (deterministic) algorithm that solves this problem in $O(\ell^3 N \log N)$ time or better.

[Hint: use divide-and-conquer, like in the midterm 1 practice question. Recall that for $A, B \subseteq \{1, \dots, M\}$, we can compute $A + B = \{a + b : a \in A, b \in B\}$ in $O(M \log M)$ time by convolution/FFT. It may be helpful to extend the problem to computing $g_k(S_1, \dots, S_\ell)$ for all $k \leq \ell$ simultaneously.]

- (b) (50 pts) Now design and analyze a Monte Carlo algorithm to compute $f_k(S)$. The running time should be of the form $O(k^c N \log^{c'} N)$ for some constants c, c' , and the overall error probability should be at most $1/N$.

[Hint: partition S into ℓ disjoint sets randomly or by using a hash function, and then use part (a). How should we set ℓ ?]

Solution:

- (a) In the following pseudocode, we are given sets $S_1, \dots, S_\ell \subseteq \{1, \dots, N\}$, and want to output a sequence of new sets $\langle g_0(S_1, \dots, S_\ell), \dots, g_\ell(S_1, \dots, S_\ell) \rangle$. (In particular, we obtain $g_k(S_1, \dots, S_\ell)$.)

```

computeG( $S_1, \dots, S_\ell$ ):
1.  if  $\ell = 1$  then return  $\langle \{0\}, S_1 \rangle$ 
2.   $\langle G'_0, \dots, G'_{\lfloor \ell/2 \rfloor} \rangle \leftarrow \text{computeG}(S_1, \dots, S_{\lfloor \ell/2 \rfloor})$ 
3.   $\langle G''_0, \dots, G''_{\lfloor \ell/2 \rfloor} \rangle \leftarrow \text{computeG}(S_{\lfloor \ell/2 \rfloor + 1}, \dots, S_\ell)$ 
4.  for  $k = 0, \dots, \ell$  do
5.     $G_k = \bigcup_{j=0}^k (G'_j + G''_{k-j})$ 
6.  return  $\langle G_0, \dots, G_\ell \rangle$ 

```

(In line 5, we take G'_j, G''_{k-j} as $\emptyset$ if indices are out-of-bound.)

Correctness. Any element $z \in g_k(S_1, \dots, S_\ell)$ can be expressed as a sum $a + b$ with $a \in g_j(S_1, \dots, S_{\lfloor \ell/2 \rfloor})$ and $b \in g_{k-j}(S_{\lfloor \ell/2 \rfloor + 1}, \dots, S_\ell)$ for some j .

Run time analysis. Computing $G'_j + G''_{k-j}$ reduces to convolution, as mentioned, and so takes $O(\ell N \log N)$ time, since $G'_j, G''_{k-j} \subseteq \{0, \dots, \ell N\}$. Thus, line 5 takes $O(\ell^2 N \log N)$ time. Lines 4–5 take $O(\ell^3 N \log N)$. Let $T(\ell, N)$ denote the running time of $\text{computeG}(S_1, \dots, S_\ell)$ with $S_1, \dots, S_\ell \subseteq \{1, \dots, N\}$. We obtain the following recurrence:

$$T(\ell, N) = 2T(\ell/2, N) + O(\ell^3 N \log N).$$

The recurrence solves to $T(\ell, N) = O(\ell^3 N \log N)$ (since the related recurrence $T'(\ell) = 2T'(\ell/2) + O(\ell^3)$ solves to $T'(\ell) = O(\ell^3)$ by the master theorem).

[*Improvement.* We could alternatively compute all G_k 's simulatenously by *one* convolution but over two longer vectors with $O(\ell \cdot \ell N)$ length. This reduces the run time of lines 4–5 to $O(\ell^2 N \log N)$. So, the total run time is $O(\ell^2 N \log N)$.]

[*Another improvement.* If we take k into account and change the algorithm to compute $\langle g_0(S_1, \dots, S_\ell), \dots, g_k(S_1, \dots, S_\ell) \rangle$, the recurrence can be further improved to $T(k, \ell, N) = 2T(\min\{k, \ell/2\}, \ell/2, N) + O(k^2 N \log N)$, which actually gives $O(k\ell N \log N) \dots$]

- (b) *Pseudocode.* Let d and ℓ be parameters to be chosen later.

```

0.  ANS =  $\emptyset$ 
1.  repeat  $d$  times {
2.    for each  $x \in S$ , pick a random number  $h(x) \in \{1, \dots, \ell\}$  and insert  $x$  to  $S_{h(x)}$ 
3.    ANS = ANS  $\cup g_k(S_1, \dots, S_\ell)$ 
  }
4.  return ANS

```

Error probability analysis. It is clear that $\text{ANS} \subseteq f_k(S)$ (since in each iteration, $S_1, \dots, S_\ell$ are disjoint and so $g_k(S_1, \dots, S_\ell) \subseteq f_k(S)$).

For the reverse direction, fix $z \in f_k(S)$. Write $z = a_1 + \dots + a_k$ for distinct numbers $a_1, \dots, a_k \in S$. Consider one iteration of the repeat loop. If $h(a_1), \dots, h(a_k)$ are all different, then $a_1, \dots, a_k$ belong to k different sets among $S_1, \dots, S_\ell$ and so $z \in g_k(S_1, \dots, S_\ell)$. Thus,

$$\begin{aligned} \Pr_h[z \notin g_k(S_1, \dots, S_\ell)] &\leq \Pr_h[h(a_i) = h(a_j) \text{ for some } i, j \in \{1, \dots, k\} \text{ (} i \neq j \text{)}] \\ &\leq \sum_{i, j \in \{1, \dots, k\} \text{ (} i \neq j \text{)}} \Pr_h[h(a_i) = h(a_j)] \quad (\text{by the union bound}) \\ &= \sum_{i, j \in \{1, \dots, k\} \text{ (} i \neq j \text{)}} \frac{1}{\ell} \\ &\leq \frac{k^2}{\ell}. \end{aligned}$$

(This is similar to an analysis from class on perfect hashing, or the “birthday paradox”.) After all d iterations, we have

$$\Pr_h[z \notin \text{ANS}] \leq \left(\frac{k^2}{\ell}\right)^d$$

by independence. Thus, by applying the union bound over all $z \in \{1, \dots, kN\}$, we can upper-bound the overall error probability (i.e., probability that $\text{ANS} \neq f_k(S)$) by

$$kN \cdot \left(\frac{k^2}{\ell}\right)^d.$$

We can choose $\ell = 2k^2$ and $d = 2\log_2(kN)$ and this probability is less than $kN \cdot (1/2)^{2\log_2(kN)} \leq 1/(kN)$ as desired.

Running time analysis. The computation of $g_k(S_1, \dots, S_\ell)$ in line 3 takes $O(\ell^3 N \log N)$ time by part (a). Thus, the total time over all d iterations is $O(d\ell^3 N \log N)$. Since we choose $\ell = 2k^2$ and $d = 2\log_2(kN) = O(\log N)$, the final time bound is $O(k^6 N \log^2 N)$.

[*Remark.* In the above, we choose h completely randomly. Alternatively, we may choose h more specifically from a universal family of hash functions $h_{a,b}$ as given in class (so instead of picking n random numbers, we just need two random numbers a, b per iteration).]

[*Improvement.* With the $O(k\ell N \log N)$ improved version of part (a), the final time bound improves to $O(k^3 N \log^2 N)$.]

[*Remark.* There is actually a still faster algorithm by Koiliaris and Xu (2017) that computes $f_k(S)$ in $O(k^2 N \log^2 N)$ time, without needing randomization at all!]

Problem 5.3: Consider the following geometric problem: given a set P of n points in 2D, with integer coordinates from $\{0, 1, \dots, U-1\}$, find a *closest pair*, i.e., two points $p, q \in P$ ($p \neq q$) with the smallest Euclidean distance. Let $\delta(P)$ denote the distance of the closest pair.

We have seen an $O(n \log n)$ -time divide-and-conquer algorithm from class. In this question, we give a different, faster randomized algorithm (which has the added advantage that it can be extended to higher dimensions).

- (a) (35 pts) First give an $O(n)$ -expected-time (Las Vegas) algorithm for the easier *decision problem*: given a value r , decide whether $\delta(P) < r$.

(Hints: Build a uniform grid where each cell is an $(r/2) \times (r/2)$ square. Use hashing. How many points can a grid cell have? For each grid cell, how many grid cells are of distance at most r ?)

- (b) (65 pts) Now, consider the following recursive Las Vegas algorithm to compute $\delta(P)$:

CLOSEST-PAIR(P):

1. if $|P| \leq 100$ then return answer by brute force
2. partition P into subsets $P_1, \dots, P_{20}$ each with at most $\lceil n/20 \rceil$ points
3. let $S = \{(i, j) \mid 1 \leq i < j \leq 20\}$
4. $r = \infty$
5. for each $(i, j) \in S$ in random order do
6. if $\delta(P_i \cup P_j) < r$ then
7. $r = \text{CLOSEST-PAIR}(P_i \cup P_j)$
8. return r

Explain why the algorithm is always correct, and analyze its expected running time by solving a recurrence.

(Hints: Where is part (a) used? What is $|S|$? What is the expected number of times line 7 is performed, using facts from class?)

Solution:

- (a) Given a value r and a set of n points $p_1, \dots, p_n$ where $p_i = (x_i, y_i)$, our algorithm proceeds as follows:

1. store the input point set in a dictionary D where each point p_i has key $\left\lfloor \frac{x_i}{r/2} \right\rfloor U + \left\lfloor \frac{y_i}{r/2} \right\rfloor$
2. for each point p_i do
3. let $A = \left\lfloor \frac{x_i}{r/2} \right\rfloor$ and $B = \left\lfloor \frac{y_i}{r/2} \right\rfloor$
4. for $a = A - 2$ to $A + 2$ do
5. for $b = B - 2$ to $B + 2$ do
6. if $aU + b$ is the key of some point p_j in D then
7. if p_i and p_j have distance $< r$ then return “yes”
8. return “no”

Correctness: Consider a grid where each cell is an $(r/2) \times (r/2)$ square.

Suppose two points p_i and p_j are in the same cell. Then their distance is at most $\sqrt{2} \cdot r/2 < r$, implying that $\delta(P) < r$. Since the two points have the same key, line 7 would correctly return yes in the iteration for p_i for $a = A$ and $b = B$.

Now suppose that each cell contains at most one point. If no pair has distance $< r$, then line 10 would return no. Suppose there exists a pair p_i, p_j of distance $< r$. Say p_i and p_j must be in nearby cells in the grid. More precisely, the column containing p_j must be next to either the column containing p_i or a column next to that column (because each column has width $r/2$). Similarly, the row containing p_j must be next to either the row containing p_i or a row next to that row. Thus, line 9 would find the pair and return yes.

Running time: With the hashing method from class, line 6 takes $O(1)$ expected time, after $O(n)$ -time preprocessing in line 1. The number of times line 6 is executed is at most $25n$. So, the total expected time is $O(n)$.

- (b) We observe that $\delta(P) = \min_{1 \leq i < j \leq 20} \delta(P_i \cup P_j)$: we have $\leq$ because the closest pair must lie inside $P_i \cup P_j$ for some i, j ; we have $\geq$ because clearly $\delta(P_i \cup P_j) \leq \delta(P)$ for all i, j . It follows (by induction) that Closest-Pair(P) returns the minimum of $\delta(P_i \cup P_j)$ over all i, j , which is $\delta(P)$.

Note that $|S| = \binom{20}{2} = 190$. By a result from class (the 1D subproblem considered in the lecture on smallest enclosing circles), the expected number of times that line 7 is performed is given by the Harmonic number $1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{190} < \ln 190 + 1 < 7$, for example, by Cormen et al.'s textbook [p. 1156 (3rd ed.)].

The size of each $P_i \cup P_j$ is at most $\lceil n/10 \rceil$. Therefore, the expected running time satisfies the recurrence

$$T(n) < 7T(\lceil n/10 \rceil) + O(n),$$

since line 6 takes $O(n)$ time by part (a). By the master theorem (or expanding the recurrence into a geometric series), we see that $T(n) = O(n)$.

(Note: The same algorithm works in any constant dimension. The first known randomized linear-time algorithm for the closest pair problem was given by Rabin (1976). The approach here is from this paper of mine.)